

Python: exercises on dictionaries, tuples, and sets

This notebook was generated in pseudocode mode.

Exercise 1: Merge Two Dictionaries

Exercise Description

Create a function `merge_dicts(dict1, dict2)` that combines two dictionaries. If a key exists in both, sum their values.

Your solution

```
def merge_dicts(dict1, dict2):  
    # Step 1: Create a copy of the first dictionary to avoid modifying the original  
    # Step 2: Iterate through each key-value pair in the second dictionary  
    # Step 3: For each key in the second dictionary:
```

```
# - If the key exists in the result, add the values together
# - If the key doesn't exist, add the key-value pair to the result
# Step 4: Return the merged dictionary
```

Test your solution

```
# Test case 1
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}
result = merge_dicts(dict1, dict2)
print(result) # Expected: {'a': 1, 'b': 5, 'c': 4}
```

Test your solution

```
# Test case 2
dict1 = {'x': 10}
dict2 = {'y': 20, 'x': 5}
result = merge_dicts(dict1, dict2)
print(result) # Expected: {'x': 15, 'y': 20}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 2: Reverse Lookup

Exercise Description

Write a function `reverse_lookup(d, value)` that returns all keys from dictionary `d` that map to the given `value`.

Your solution

```
def reverse_lookup(d, value):  
    # Step 1: Initialize an empty list to store matching keys  
    # Step 2: Iterate through each key-value pair in the dictionary  
    # Step 3: For each pair, if the value matches the target value:  
    #         - Add the key to the result list  
    # Step 4: Return the list of matching keys
```

Test your solution

```
# Test case 1  
d = {'a': 1, 'b': 2, 'c': 1}  
result = reverse_lookup(d, 1)  
print(result) # Expected: ['a', 'c']
```

Test your solution

```
# Test case 2
d = {'x': 'hello', 'y': 'world', 'z': 'hello'}
result = reverse_lookup(d, 'hello')
print(result) # Expected: ['x', 'z']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 3: Count Tuple Elements

Exercise Description

Write a function `count_elements(tup)` that returns a dictionary with counts of each element in the tuple `tup`.

Your solution

```
def count_elements(tup):
    # Step 1: Initialize an empty dictionary to store counts
```

```
# Step 2: Iterate through each element in the tuple
# Step 3: For each element:
#     - If element exists in dictionary, increment its count
#     - If element doesn't exist, set its count to 1
# Step 4: Return the dictionary with element counts
```

Test your solution

```
# Test case 1
tup = (1, 2, 2, 3, 1)
result = count_elements(tup)
print(result) # Expected: {1: 2, 2: 2, 3: 1}
```

Test your solution

```
# Test case 2
tup = ('a', 'b', 'a', 'c', 'b', 'a')
result = count_elements(tup)
print(result) # Expected: {'a': 3, 'b': 2, 'c': 1}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 4: Set Intersection

Exercise Description

Implement a function `intersect(s1, s2)` that returns the intersection of two sets `s1` and `s2`.

Your solution

```
def intersect(s1, s2):  
    # Step 1: Initialize an empty set to store common elements  
    # Step 2: Iterate through each element in the first set  
    # Step 3: For each element, if it's also in the second set:  
    #         - Add it to the result set  
    # Step 4: Return the intersection set  
    # Alternative: Use built-in set intersection operation (s1 & s2)
```

Test your solution

```
# Test case 1  
s1 = {1, 2, 3}  
s2 = {2, 3, 4}  
result = intersect(s1, s2)  
print(result) # Expected: {2, 3}
```

Test your solution

```
# Test case 2
s1 = {'a', 'b', 'c'}
s2 = {'b', 'c', 'd'}
result = intersect(s1, s2)
print(result) # Expected: {'b', 'c'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 5: Remove Duplicates

Exercise Description

Write a function `remove_duplicates(lst)` that returns a list without any duplicates using a set.

Your solution

```
def remove_duplicates(lst):  
    # Step 1: Convert the list to a set (automatically removes duplicates)  
    # Step 2: Convert the set back to a list  
    # Step 3: Return the list without duplicates  
    # Note: Order may not be preserved
```

Test your solution

```
# Test case 1  
lst = [1, 2, 2, 3, 4, 4, 5]  
result = remove_duplicates(lst)  
print(result) # Expected: [1, 2, 3, 4, 5]
```

Test your solution

```
# Test case 2  
lst = ['a', 'b', 'a', 'c', 'b']  
result = remove_duplicates(lst)  
print(result) # Expected: ['a', 'b', 'c'] (order may vary)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 6: Tuple to Dictionary

Exercise Description

Create a function `tuple_to_dict(tup)` that turns a tuple of `(key, value)` pairs into a dictionary.

Your solution

```
def tuple_to_dict(tup):  
    # Step 1: Initialize an empty dictionary  
    # Step 2: Iterate through each pair in the tuple  
    # Step 3: For each pair (key, value):  
    #     - Add the key-value pair to the dictionary  
    # Step 4: Return the dictionary  
    # Alternative: Use dict() constructor directly
```

Test your solution

```
# Test case 1  
tup = (('a', 1), ('b', 2))  
result = tuple_to_dict(tup)  
print(result) # Expected: {'a': 1, 'b': 2}
```

Test your solution

```
# Test case 2
tup = (('x', 10), ('y', 20), ('z', 30))
result = tuple_to_dict(tup)
print(result) # Expected: {'x': 10, 'y': 20, 'z': 30}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 7: Value Switch in Dictionary

Exercise Description

Write a function `switch_values(d)` that switches the keys and values in a dictionary. Assume all values are unique.

Your solution

```
def switch_values(d):
    # Step 1: Initialize an empty dictionary for the result
```

```
# Step 2: Iterate through each key-value pair in the original dictionary
# Step 3: For each pair:
#     - Use the original value as the new key
#     - Use the original key as the new value
# Step 4: Return the switched dictionary
```

Test your solution

```
# Test case 1
d = {'a': 1, 'b': 2}
result = switch_values(d)
print(result) # Expected: {1: 'a', 2: 'b'}
```

Test your solution

```
# Test case 2
d = {'name': 'John', 'age': 25}
result = switch_values(d)
print(result) # Expected: {'John': 'name', 25: 'age'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 8: Dictionary of Squares

Exercise Description

Create a function `squares(n)` that returns a dictionary of numbers from 1 to `n` where the keys are numbers and the values are their squares.

Your solution

```
def squares(n):  
    # Step 1: Initialize an empty dictionary  
    # Step 2: Iterate through numbers from 1 to n (inclusive)  
    # Step 3: For each number:  
    #     - Use the number as the key  
    #     - Use the square of the number as the value  
    # Step 4: Return the dictionary
```

Test your solution

```
# Test case 1  
result = squares(5)  
print(result) # Expected: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Test your solution

```
# Test case 2
result = squares(3)
print(result) # Expected: {1: 1, 2: 4, 3: 9}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 9: Set Difference

Exercise Description

Write a function `difference(s1, s2)` that returns a set of elements that are only in the first set `s1` but not in the second set `s2`.

Your solution

```
def difference(s1, s2):
    # Step 1: Initialize an empty set for the result
    # Step 2: Iterate through each element in the first set
```

```
# Step 3: For each element, if it's NOT in the second set:  
#     - Add it to the result set  
# Step 4: Return the difference set  
# Alternative: Use built-in set difference operation (s1 - s2)
```

Test your solution

```
# Test case 1  
s1 = {1, 2, 3, 4}  
s2 = {3, 4, 5, 6}  
result = difference(s1, s2)  
print(result) # Expected: {1, 2}
```

Test your solution

```
# Test case 2  
s1 = {'a', 'b', 'c'}  
s2 = {'b', 'd'}  
result = difference(s1, s2)  
print(result) # Expected: {'a', 'c'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 10: Count Vowels

Exercise Description

Create a function `count_vowels(s)` that counts and returns a dictionary with vowels as keys and their counts in the string `s` as values.

Your solution

```
def count_vowels(s):  
    # Step 1: Define a set of vowels (a, e, i, o, u)  
    # Step 2: Initialize an empty dictionary to store vowel counts  
    # Step 3: Iterate through each character in the string (convert to lowercase)  
    # Step 4: For each character, if it's a vowel:  
    #         - If vowel exists in dictionary, increment its count  
    #         - If vowel doesn't exist, set its count to 1  
    # Step 5: Return the vowel count dictionary
```

Test your solution

```
# Test case 1  
result = count_vowels('hello world')
```

```
print(result) # Expected: {'e': 1, 'o': 2}
```

Test your solution

```
# Test case 2
result = count_vowels('programming')
print(result) # Expected: {'o': 1, 'a': 1, 'i': 1}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 11: Find Max Value

Exercise Description

Write a function `find_max(d)` that returns the key with the maximum value in the dictionary `d`.

Your solution

```
def find_max(d):  
    # Step 1: Initialize variables to track the maximum value and corresponding key  
    # Step 2: Iterate through each key-value pair in the dictionary  
    # Step 3: For each pair, if this value is greater than the current maximum:  
    #         - Update both the maximum value and the corresponding key  
    # Step 4: Return the key with the maximum value  
    # Alternative: Use max() function with key parameter
```

Test your solution

```
# Test case 1  
d = {'a': 5, 'b': 12, 'c': 3}  
result = find_max(d)  
print(result) # Expected: 'b'
```

Test your solution

```
# Test case 2  
d = {'x': 100, 'y': 50, 'z': 200}  
result = find_max(d)  
print(result) # Expected: 'z'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 12: Unique Characters

Exercise Description

Create a function `unique_chars(s)` that returns a set of unique characters in the string `s`.

Your solution

```
def unique_chars(s):  
    # Step 1: Convert the string to a set (automatically removes duplicates)  
    # Step 2: Return the set of unique characters
```

Test your solution

```
# Test case 1  
result = unique_chars('hello')  
print(result) # Expected: {'e', 'h', 'l', 'o'}
```

Test your solution

```
# Test case 2  
result = unique_chars('programming')
```

```
print(result) # Expected: {'p', 'r', 'o', 'g', 'a', 'm', 'i', 'n'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 13: Key Multiplication

Exercise Description

Write a function `multiply_keys(d, factor)` that returns a new dictionary where each key is multiplied by a `factor`.

Your solution

```
def multiply_keys(d, factor):  
    # Step 1: Initialize an empty dictionary for the result  
    # Step 2: Iterate through each key-value pair in the original dictionary  
    # Step 3: For each pair:  
    #         - Multiply the key by the factor
```

```
# - Use the multiplied key with the original value in the result dictionary
# Step 4: Return the new dictionary
```

Test your solution

```
# Test case 1
d = {1: 100, 2: 200}
result = multiply_keys(d, 2)
print(result) # Expected: {2: 100, 4: 200}
```

Test your solution

```
# Test case 2
d = {3: 'a', 5: 'b'}
result = multiply_keys(d, 10)
print(result) # Expected: {30: 'a', 50: 'b'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 14: Set Complement

Exercise Description

Implement a function `complement(full_set, sub_set)` that returns the set of elements in `full_set` that are not in `sub_set`.

Your solution

```
def complement(full_set, sub_set):  
    # Step 1: Initialize an empty set for the result  
    # Step 2: Iterate through each element in the full set  
    # Step 3: For each element, if it's NOT in the subset:  
    #     - Add it to the result set  
    # Step 4: Return the complement set  
    # Alternative: Use built-in set difference operation (full_set - sub_set)
```

Test your solution

```
# Test case 1  
full_set = {1, 2, 3, 4, 5}  
sub_set = {2, 4}  
result = complement(full_set, sub_set)  
print(result) # Expected: {1, 3, 5}
```

Test your solution

```
# Test case 2
full_set = {'a', 'b', 'c', 'd'}
sub_set = {'b', 'd'}
result = complement(full_set, sub_set)
print(result) # Expected: {'a', 'c'}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.'
```

Exercise 15: Count Keys Starting With

Exercise Description

Write a function `count_keys_starting_with(d, char)` that returns the count of keys in dictionary `d` that start with `char`.

Your solution

```
def count_keys_starting_with(d, char):  
    # Step 1: Initialize a counter to 0  
    # Step 2: Iterate through each key in the dictionary  
    # Step 3: For each key:  
    #     - Convert key to string (in case it's not already a string)  
    #     - Check if the first character matches the target character  
    #     - If it matches, increment the counter  
    # Step 4: Return the counter
```

Test your solution

```
# Test case 1  
d = {'apple': 1, 'banana': 2, 'apricot': 3, 'cherry': 4}  
result = count_keys_starting_with(d, 'a')  
print(result) # Expected: 2
```

Test your solution

```
# Test case 2  
d = {'hello': 1, 'world': 2, 'python': 3}  
result = count_keys_starting_with(d, 'h')  
print(result) # Expected: 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the

